

Exercise: DDL, Constraints & Inserts, Updates, Deletes

Schema: FleetDesk — a campus IT service desk tracking devices, departments, technicians, alerts, and work orders.

You are the on-call security apprentice for FleetDesk, a campus IT service desk system. The schema below was written quickly to get the app shipped, and it compiles cleanly — but “it runs” is not “it is safe.” Your job in this exercise is the same job from the reading: read what's missing, add the constraints that make it defensible, write a safe transactional change, build a view for the dashboards asking for one, and catch an unbounded write before it ships.

Show your SQL for every part that asks for it. Written-answer parts should be 2–4 sentences unless noted otherwise.

The FleetDesk Schema (as shipped)

Five tables. Read each one the way you'd read a script someone else wrote before running it against production — what's required, what's related to what, and what's missing.

devices — laptops, desktops, and loaner hardware

```
CREATE TABLE devices (  
  device_id INT GENERATED ALWAYS AS IDENTITY,  
  asset_tag VARCHAR(32),  
  device_type VARCHAR(20),  
  location VARCHAR(64),  
  status VARCHAR(20)  
);
```

departments — campus departments FleetDesk supports

```
CREATE TABLE departments (  
  department_id INT GENERATED ALWAYS AS IDENTITY,  
  name VARCHAR(64),  
  building VARCHAR(64),  
  support_tier VARCHAR(20)  
);
```

technicians — service desk staff

```
CREATE TABLE technicians (  
  technician_id INT GENERATED ALWAYS AS IDENTITY,  
  username VARCHAR(32),  
  team VARCHAR(64),  
  role VARCHAR(32),  
  active BOOLEAN  
);
```

alerts — automated monitoring alerts against a device

```
CREATE TABLE alerts (  
  alert_id INT GENERATED ALWAYS AS IDENTITY,  
  device_id INT,  
  department_id INT,  
  severity VARCHAR(20),  
  message VARCHAR(400),  
  occurred_at TIMESTAMP  
);
```

work_orders — service desk tickets: incidents and change requests

```
CREATE TABLE work_orders (  
  work_order_id INT GENERATED ALWAYS AS IDENTITY,  
  device_id INT,  
  department_id INT,  
  kind VARCHAR(20),  
  status VARCHAR(20),  
  priority VARCHAR(10),  
  opened_at TIMESTAMP,  
  resolved_at TIMESTAMP,  
  assignee_technician_id INT  
);
```

ALLOWED VALUE SETS (for any CHECK you write)

devices.status: 'active', 'repair', 'retired', 'lost' • alerts.severity: 'info', 'warning', 'error', 'critical' •
work_orders.priority: 'P1', 'P2', 'P3', 'P4' • work_orders.status: 'open', 'in_progress', 'resolved', 'closed' •
departments.support_tier: 'standard', 'priority', 'executive'

Part A — Read the Schema, Then Constrain It

A1. List every constraint missing from the devices table above — name the constraint type for each gap (not SQL yet). Don't forget to consider whether every column should be required.

A2. Write the ALTER TABLE statements to fully constrain devices: a primary key on device_id, NOT NULL on every column that should be required, a UNIQUE constraint on asset_tag, and a CHECK restricting status to the allowed value set above.

A3. Write the ALTER TABLE statements to fully constrain alerts: a primary key, foreign keys to devices and departments, a CHECK on severity, NOT NULL on message, and a DEFAULT for occurred_at. For the two foreign keys, decide whether each should be NOT NULL or nullable — and be ready to justify that choice in Part B's spirit (an alert with no device is not a state FleetDesk should allow to exist).

Part B — The Nullable-FK Judgment

B1. work_orders.assignee_technician_id should be a nullable foreign key to technicians. Argue why NULL is a legitimate domain state for this column — what does NULL correctly represent here?

B2. Name one other column on work_orders where NULL would instead be a defect — a column you would lock out with NOT NULL — and justify why that column can never legitimately be empty.

Part C — The Cascade Tradeoff

C1. Choose an ON DELETE action for the foreign key from alerts to devices, for the case where a device is permanently decommissioned and removed from inventory. Justify your choice using the ownership test from class.

C2. Choose an ON DELETE action for the foreign key from work_orders to technicians (assignee_technician_id), for the case where a technician leaves the service desk team. Justify your choice.

Part D — A Transactional Change

Leadership wants a new `sla_breached` column on `work_orders`: a `BOOLEAN`, `NOT NULL`, defaulting to `false`. Every currently-open P1 work order that was opened before 2026-01-01 should be flagged `sla_breached = true` on rollout. The new dashboard also needs an index on the column.

D1. Write the full script — `ALTER TABLE`, `UPDATE`, and `CREATE INDEX` — wrapped in a transaction so the change is safe to run in a single change window.

D2. In 2–3 sentences: if the `UPDATE` statement failed partway through (say, a typo in the date literal), what would `ROLLBACK` leave behind, and why does that matter operationally compared to not using a transaction at all?

Part E — Views

Three separate dashboards, plus a Monday-morning email report, all need the same number: a count of open P1 work orders, broken down by department name.

E1. Write the `CREATE VIEW` statement that produces this shape, so every consumer can just `SELECT *` FROM it.

E2. A different analyst wants to run a one-off historical query — total resolution time for every work order closed last quarter, for a single audit that will not repeat. Should that query become a view, or stay a saved query? Justify using the decision criteria from class.

Part F — Spot the Unbounded Write

A teammate is about to run the following statement to close out a single resolved ticket, `work_order_id` 4521:

```
UPDATE work_orders SET status = 'resolved';
```

F1. What actually happens if this statement runs as written?

F2. Rewrite the statement correctly, and describe the discipline you'd use to double-check it before running it against production.

Part G — Reflection

Pick one constraint you added in Part A or Part C. In 3–5 sentences, describe a specific bad write that could land in FleetDesk today without it — imagine a second writer (a new integration, a bulk import tool, a script run by a new hire) that never goes through the service desk's front-end validation. Explain exactly how your constraint would refuse that write.

Part H — Basic DML From Scratch

Write each statement below from scratch — no template is given. Assume every constraint from Parts A–C is already in place.

H1. A new laptop arrives with asset tag 'AT-2044', device type 'laptop', location 'Library 2F', and status 'active'. Write the INSERT statement that adds it to devices.

H2. Technician jmiller is going on extended leave. Write the UPDATE statement that marks `active = false` for the technician whose username is 'jmiller'.

H3. A duplicate department row — name 'IT Support (temp)', building 'Annex' — was created by mistake and has no alerts or work orders referencing it. Write the DELETE statement that removes exactly that row.

SUBMISSION

Submit your SQL and written answers as a single document, in the same part order (A–H). Label every statement with the part and item it answers (e.g. “A2”) so your work can be matched against the rubric.